

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

ФАКУЛТЕТ ПО КОМПЮТЪРНИ СИСТЕМИ И ТЕХНОЛОГИИ

КУРСОВ ПРОЕКТ

по

СИГУРНИ ПРОТОКОЛИ И АРХИТЕКТУРИ ЗА КОМУНИКАЦИИ

Студент: Алекс Иванов Цветанов
Група: 263 Фак. №: 371326005

ТЕМА

Разпознаване на TLS и некриптиран HTTP по първия октет при един слушащ сокет: механизъм и анализ на сигурността

Ръководител: _____

(проф. Георги Илиев)

София, 2026

Съдържание

1. Въведение 4

1.1. Какво е направено 4

1.2. Какво работата твърди и какво не 5

1.3. Структура на работата 6

2. Протоколна основа 6

2.1. Записният слой на TLS 6

2.2. Ръкостискането в TLS 1.3 7

2.3. Предупрежденията 7

2.4. Договаряне на приложния протокол 8

2.5. Сертификати и верига на доверие 8

2.6. Началото на HTTP заявката 9

2.7. Защо един октет е достатъчен 9

3. Начини за споделяне на един порт 10

3.1. Отделен порт за всеки протокол 10

3.2. Надграждане на некриптирана връзка 10

3.3. Опцията port-share на OpenVPN 11

3.4. sslh 11

3.5. Сравнение 12

4. Механизмът в octet 12

4.1. Пътят на една връзка 12

4.2. Разпознаването 14

4.3. Връщане на прочетените октети 15

4.4. OpenSSL върху буфери в паметта 16

4.5. Един общ краен срок 17

4.6. Настройките на TLS контекста 18

4.7. Тестовият удостоверяващ орган 19

5. Анализ на сигурността 20

5.1. Модел на заплахите 20

5.2. Заплахи и отговори 20

5.3. Отклонение от препоръка в HTTP/1.1 22

5.4. Грешката, която тестовете откриха 22

5.5. Атаки с понижаване на защитата 23
5.6. Какво не е проверено 24

6. Проверка **24**
6.1. Тестове на ниво функция 25
6.2. TLS сесия в паметта 25
6.3. Интеграционен тест 26
6.4. Какво остава непроверено 28

7. Измерване на цената **28**
7.1. Постановка 28
7.2. Статистика 29
7.3. Първото измерване и какво показа то 30
7.4. Резултати 30
7.5. Цената в паметта 31
7.6. Тълкуване 32
7.7. Ограничения на измерването 32

8. Изводи и заключение **33**

Списък с използваните съкращения **34**

Списък с използваните литературни източници **35**

1. Въведение

Уеб сървърът традиционно слуша на два порта. На порт 80 приема некриптиран HTTP, а на порт 443 приема HTTP, защитен с TLS [1]. Двата порта са две отделни входни точки и всяка от тях знае предварително какъв протокол ще пристигне. Сървърът, който слуша на 443, започва TLS ръкостискане веднага след приемането на връзката, без да проверява нищо. Сървърът на 80 подава байтовете направо на анализатора на HTTP заявки.

Тази работа разглежда обратния подход. Сървърът има един-единствен слушащ сокет и за всяка приета връзка сам решава дали тя е TLS или некриптиран HTTP. Решението се взема по първия октет, който клиентът изпраща. Това е възможно, защото първият запис на всеки TLS клиент започва с октет 0x16, а всяка HTTP заявка започва с главна латинска буква. Двете множества не се пресичат, така че един октет е достатъчен.

Темата попада в дисциплината по три линии. Първо, механизмът стъпва директно на структурата на записния слой на TLS [2]: без точното значение на полето ContentType разпознаването е невъзможно. Второ, решението „TLS или не“ се взема преди ръкостискането, тоест в момент, когато връзката още не е защитена с нищо. Този прозорец има собствени заплахи и те са основният предмет на анализа. Трето, след разпознаването сървърът провежда истинско TLS ръкостискане със сертификат, верига от сертификати, договаряне на приложния протокол (ALPN [3]) и ограничения на версията. Всяка от тези настройки има последствия за сигурността.

1.1. Какво е направено

Идеята произхожда от уеб сървър, който авторът разработва. За целите на проекта механизмът е реализиран наново, самостоятелно и от нулата, в отворено хранилище с име `octet` [4]. Кодът е на C++23 и използва OpenSSL. Сървърът е проверен на Windows (MSVC, OpenSSL 3.6.4) и на Linux (GCC 14, OpenSSL

3.0.13). За macOS има конфигурация за компилиране, но тя не е проверена и работата не твърди нищо за тази платформа.

Конкретно проектът съдържа:

- сървър, който приема TLS и некриптиран HTTP/1.1 на един порт и разпознава протокола по първия октет;
- пренасочване на прочетените при разпознаването октети обратно към TLS библиотеката, без загуба на нито един байт;
- един общ краен срок, който покрива и разпознаването, и цялото TLS ръкостискане;
- строго договаряне на ALPN, минимална версия TLS 1.2 или 1.3, забранено предоговаряне и забранена компресия;
- собствен удостоверяващ орган за тестове и сертификат на сървъра, подписан от него;
- автоматични тестове на ниво функция, на ниво TLS сесия в паметта и на ниво изпълним файл през истински сокети;
- измерване на цената на разпознаването на две платформи.

1.2. Какво работата твърди и какво не

Работата твърди три неща и всяко от тях е проверено с тест или с измерване. Първо, един октет разделя TLS от HTTP без грешка за всичките 256 възможни стойности. Второ, сървърът с разпознаване обслужва и двата транспорта на един порт и отхвърля всичко останало. Трето, на машината, на която е измерено, цената на разпознаването е под разделителната способност на измерването от край до край, а в паметта тя е под една микросекунда на връзка.

Работата не твърди, че така изграденият сървър е готов за производствена среда. В раздел 5 се показва, че самото съчетаване на защитен и незащитен транспорт на един порт отваря път за атаки с понижаване на защитата. Това е известен проблем, описан в препоръките на IETF [5], и работата го разглежда открито, вместо да го заобикаля.

1.3. Структура на работата

Раздел 2 описва протоколната основа: записния слой на TLS, ръкостискането, ALPN, сертификатите и началото на HTTP заявката. Раздел 3 сравнява известни начини за споделяне на един порт между няколко протокола. Раздел 4 описва механизма в `octet` с откъси от кода. Раздел 5 е анализът на сигурността. Раздел 6 описва проверката, а раздел 7 измерването. Раздел 8 съдържа изводите.

2. Протоколна основа

2.1. Записният слой на TLS

TLS пренася всичко, което изпраща, в записи. Всеки запис започва с петоктетна заглавна част, а след нея идва съдържанието. В TLS 1.3 структурата се нарича `TLSPlaintext` и е определена в раздел 5.1 на RFC 8446 [2]. В TLS 1.2 същата структура е описана в раздел 6.2.1 на RFC 5246 [6]. Полетата са следните.

Таблица 1. Заглавна част на TLS запис

Поле	Октети	Съдържание
`type` (ContentType)	1	вид на съдържанието: change_cipher_spec (20), alert (21), handshake (22) или application_data (23)
`legacy_record_version`	2	в TLS 1.3 трябва да е 0x0303; в първия ClientHello може да е и 0x0301 заради съвместимост
`length`	2	дължина на съдържанието в октети; не може да надхвърля 2^{14}
`fragment`	`length`	самото съдържание

Първото поле е определящо за цялата работа. Стойностите му са малко и са фиксирани: промяна на шифъра 20 (0x14), предупреждение 21 (0x15), ръкостискане 22 (0x16) и приложни данни 23 (0x17). В TLS клиентът говори пръв. Първото, което той изпраща, е съобщението ClientHello, а то е съобщение за

ръкостискане. Следователно първият октет на всяка TLS връзка е 0x16. Шестият октет пък е типът на съобщението за ръкостискане, който за ClientHello е 1.

RFC 8446 изисква полето за версия да се игнорира и задава горна граница на дължината: запис, по-дълъг от 2^{14} октета, трябва да прекъсне връзката с предупреждение record_overflow. Тези проверки са работа на TLS библиотеката. Разпознаването в `octet` не ги дублира и в раздел 5 се обяснява защо.

2.2. Ръкостискането в TLS 1.3

Пълното ръкостискане в TLS 1.3 изисква едно отиване и връщане. Клиентът изпраща ClientHello, в което предлага поддържаните версии, параметри за обмен на ключове, по желание името на сървъра (SNI) и по желание списък с приложни протоколи (ALPN). Сървърът отговаря с ServerHello и от този момент нататък всичко е шифровано: допълнителните разширения (включително избрания приложен протокол), сертификатът, подписът върху ръкостискането и съобщението Finished. Клиентът проверява сертификата и подписа и изпраща своето Finished [2].

Важно за тази работа е, че всичко преди ServerHello пътува некриптирано. Ако сървърът откаже връзката още при обработката на ClientHello, предупреждението, с което отказва, също е некриптирано и се вижда в мрежата. Това е използвано в теста на ALPN в раздел 6: там отказът се проверява по точните октети на записа.

2.3. Предупрежденията

Предупреждението (alert) е запис с ContentType 21 и съдържание от два октета: ниво и описание [2]. Ниво 2 означава фатално предупреждение, след което връзката се затваря. Описанията, които се срещат в тази работа, са три. no_application_protocol (120) означава, че сървърът не поддържа нито един от предложените приложни протоколи. record_overflow (22) означава твърде дълъг запис. unexpected_message (10) означава съобщение, което не е на мястото си.

2.4. Договаряне на приложния протокол

TLS защитава байтове, но не казва какво означават те. Над една и съща TLS връзка може да се говори HTTP/1.1, HTTP/2 или друг протокол. Разширението ALPN решава този въпрос в самото ръкостискане [3]. Клиентът изпраща списък от имена. Всяко име е предшествано от един октет с дължината му и празни имена не са позволени (раздел 3.1 на RFC 7301). Сървърът избира едно име от списъка. Ако не поддържа нито едно, той отговаря с фаталното предупреждение no_application_protocol (раздел 3.2).

Препоръките на IETF за сигурна употреба на TLS изискват от сървърите строго да спазват това правило [5, раздел 3.8]. Причината е защитата от атаки между протоколи, при които съобщение, предназначено за един протокол, бива прието от друг. Ако сървърът приема връзката въпреки липсата на общ протокол, той дава възможност TLS сесия, предназначена за друга услуга, да бъде насочена към него.

2.5. Сертификати и верига на доверие

Сървърът доказва самоличността си със сертификат по стандарта X.509 [7]. Сертификатът свързва публичен ключ с име и е подписан от удостоверяващ орган. Клиентът има списък с органи, на които вярва. Той проверява, че веригата от сертификата на сървъра до някой от тези органи е валидна и че името в сертификата съвпада с името, към което клиентът се свързва.

В сертификата има разширения, които ограничават употребата му. Основните ограничения (basicConstraints) казват дали сертификатът е на удостоверяващ орган. Употребата на ключа (keyUsage) и разширената употреба (extendedKeyUsage) казват за какво служи ключът. Алтернативното име на субекта (subjectAltName) съдържа имената, за които сертификатът важи [7]. В раздел 4 се описва как `octet` създава за тестовете собствен орган и сертификат на сървъра с точно тези разширения.

2.6. Началото на HTTP заявката

HTTP/1.1 заявката започва с ред, съставен от метод, интервал, цел на заявката, интервал и версия на протокола [8, раздел 3]. Методът е последователност от знаци, която е чувствителна към регистъра. Всички стандартни методи са записани с главни букви по споразумение [1, раздел 9.1]: GET, HEAD, POST, PUT, DELETE, CONNECT, OPTIONS, TRACE. HTTP/2 без TLS може да започне директно с фиксирана начална последователност, която започва с `PRI * HTTP/2.0` [9, раздел 3.4]. Тя също започва с главна буква.

2.7. Защо един октет е достатъчен

От двата предходни подраздела следва, че началото на връзката попада в едно от две непресичащи се множества. TLS връзката започва с 0x16. Некриптираната HTTP заявка започва с главна латинска буква, тоест с октет между 0x41 и 0x5A. Октетът 0x16 е управляващ знак в ASCII и не е буква. Таблица 2 показва как `octet` разпределя всичките 256 стойности.

Таблица 2. Как `octet` разпределя първия октет

Първи октет	Значение	Решение
от 0x00 до 0x13	управляващи знаци	отказ
0x14, 0x15, 0x17	TLS запис, който не може да отвори връзка	отказ
0x16	TLS запис с ръкостискане, тоест ClientHello	TLS
от 0x18 до 0x40	управляващи знаци, интервал, цифри, препинателни знаци	отказ
от 0x41 до 0x5A	главни латински букви, начало на HTTP метод	HTTP
от 0x5B до 0x7F	малки букви и други знаци	отказ
от 0x80 до 0xFF	извън ASCII, включително старият формат на SSL 2.0	отказ

Две стойности заслужават отделна бележка. Записите с ContentType 20, 21 и 23 са валидни TLS записи, но никой от тях не може да е първият запис на нова връзка. Ако връзка започне с тях, другата страна не започва TLS сесия,

затова `octet` не я подава на TLS библиотеката. Старият формат на ClientHello от SSL 2.0 започва с октет със зададен най-старши бит. SSL 2.0 е забранен [10] и `octet` не го разпознава като TLS.

Малките букви също водят до отказ. Това не е пропуск: методът е чувствителен към регистъра и `get` не е методът GET. Сървър, който приема `get`, приема нещо, което не е HTTP заявка.

3. Начини за споделяне на един порт

Задачата „един порт, няколко протокола“ не е нова. Подходите се различават по това къде се взема решението, колко байта са нужни за него и кой процес провежда TLS ръкостискането. Този раздел разглежда четири подхода. За двата инструмента от трети страни се цитира само това, което пише в документацията им.

3.1. Отделен порт за всеки протокол

Това е традиционният подход: 80 за некриптиран HTTP и 443 за HTTPS [1]. Решение не се взема, защото портът вече е решението. Препоръките на IETF подкрепят отделния порт само за TLS пред динамичното надграждане на некриптирана връзка до TLS [5, раздел 3.2]. Причината е, че при отделния порт крайното състояние на канала е известно още преди първия байт.

3.2. Надграждане на некриптирана връзка

Някои протоколи започват некриптирано и по-късно преминават към TLS с команда, например STARTTLS при пощенските протоколи. RFC 9325 изброява този подход и предупреждава, че началото на връзката не е защитено и нападател може да го промени така, че надграждането да не се случи [5, раздел 3.2]. Атаката срещу командата STARTTLS е описана и в RFC 7457 [11, раздел 2.2].

3.3. Опцията port-share на OpenVPN

OpenVPN може да работи като TCP сървър. С опцията `--port-share` той споделя порта си с друго приложение, например HTTPS сървър [12]. Когато OpenVPN разпознае, че връзката не е по неговия протокол, той я препраща като посредник към зададените адрес и порт. По желание той записва във временни файлове адреса и порта на източника и на целта за всяка препратена връзка. Документацията изрично посочва, че опцията не е реализирана за Windows.

Тук решението се взема в процеса на OpenVPN. TLS ръкостискането на уеб трафика се провежда от уеб сървъра зад него, а не от OpenVPN. Цената е допълнителен скок: всяка некриптирана или HTTPS връзка минава през посредник, който копира байтовете между два сокета.

3.4. sslh

sslh е самостоятелен мултиплексор. Той приема връзки на зададени портове и ги препраща нататък според проверки върху първия пакет данни, изпратен от клиента [13]. Документацията му изброява проверки за HTTP, TLS (включително по SNI и ALPN), SSH, WireGuard, OpenVPN, tinc, XMPP и SOCKS5, както и за всеки протокол, който може да се опише с регулярен израз. Както при OpenVPN, TLS се провежда от услугата зад мултиплексора, а не от него самия.

3.5. Сравнение

Таблица 3. Сравнение на подходите за споделяне на порт

Подход	Къде се решава	Кой провежда TLS	Допълнителен скок
Отделни портове	никъде, портът е решението	сървърът на порт 443	няма
OpenVPN port-share	в процеса на OpenVPN	уеб сървърът зад него	да, посредник
sslh	в отделен процес, по първия пакет	услугата зад него	да, препращане
`octet`	в самия сървър, по първия октет	самият сървър	няма

Колко точно октета четат OpenVPN и sslh, за да вземат решение, не е посочено в цитираната документация и работата не прави предположения за това. Сравнението показва две неща. Първо, `octet` е единственият от разгледаните подходи, при който процесът, който разпознава протокола, е и процесът, който провежда ръкостискането. Няма посредник и няма втори сокет. Второ, OpenVPN и sslh различават много протоколи, а `octet` само два. Именно затова на `octet` му стига един октет. Мултиплексор, който трябва да различи SSH от OpenVPN от HTTP, не може да си позволи това.

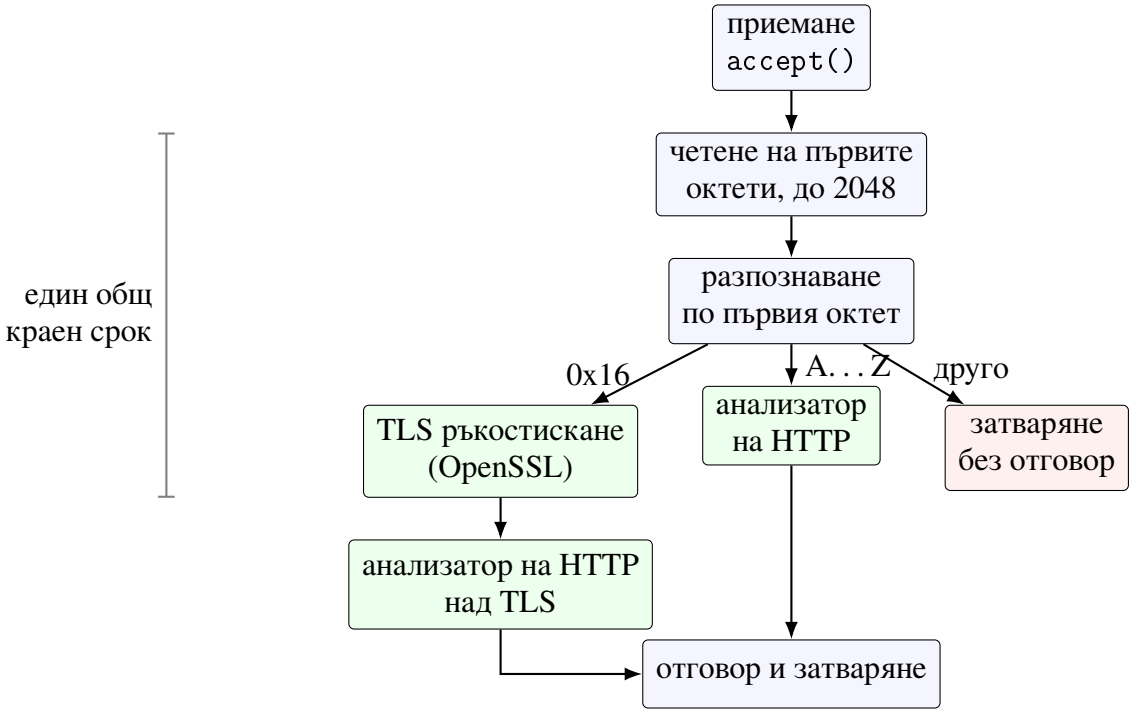
4. Механизмът в octet

Този раздел описва как `octet` обслужва една връзка, от приемането до отговора. Откъсите от кода се четат при компилиране на документа направо от изходните файлове на хранилището [4], затова текстът не може да се разминава с кода. Номерата на редовете са номерата във файла.

4.1. Пътят на една връзка

Сървърът приема връзките в един цикъл и пуска за всяка от тях отделна нишка. След това обработката минава през четири стъпки (фиг. 1). Нишката

чете първите налични октети, разпознава протокола по първия от тях и подава връзката на TLS слоя или направо на анализатора на HTTP. Всичко, което не е TLS или HTTP, се затваря без отговор.



Фигура 1. Пътят на една връзка в `octet`

Листинг 1 показва функцията, която изпълнява този път. Тя е цялата логика на демултиплексирането, около петдесет реда.

```
133     ConnectionRecord Server::handle(net::Socket sock, Clock::time_point accepted)
134     {
135         ConnectionRecord rec;
136         rec.start_us = micros_since(accepted);
137         const Deadline handshake(accepted + config_.handshake_timeout);
138
139         std::vector<std::uint8_t> first;
140         Wire wire = config_.dedicated;
141         if (config_.detect)
142         {
143             first.resize(kFirstRead);
144             const auto r = net::recv_some(sock, first, handshake);
145             if (r.status != net::Io::Ok)
146             {
147                 // Silent until the deadline, or gone before saying anything.
148                 rec.outcome = outcome_of(r.status);
149                 return rec;
150             }
151             first.resize(r.bytes);
152             rec.first_read = r.bytes;
153             wire = classify(first);
154         }
155         rec.wire = wire;
```

```
156         rec.decide_us = micros_since(accepted);
157
158         ReplayReader raw(std::move(first), [&sock](std::span<std::uint8_t> out, const
    ↪ Deadline& d) {
159             return net::recv_some(sock, out, d);
160         });
161         const tls::WriteFn raw_out = [&sock](std::span<const std::uint8_t> in, const
    ↪ Deadline& d) {
162             return net::send_all(sock, in, d);
163         };
164
165         switch (wire)
166         {
167             case Wire::Tls:
168                 serve_tls(raw, raw_out, accepted, handshake, rec);
169                 break;
170             case Wire::Cleartext:
171                 rec.ready_us = rec.decide_us;
172                 serve_http(raw, raw_out, Clock::now(), "cleartext", rec);
173                 break;
174             case Wire::Unknown:
175                 // Neither a TLS record nor an HTTP method. Nothing sensible can be said
176                 // back in a protocol the peer did not speak, so it gets no reply at all
    ↪ .
177                 rec.outcome = "unknown_protocol";
178                 break;
179         }
180
181         sock.shutdown_write();
182         return rec;  // ‘sock‘ closes here
183     }
```

Листинг 1: Обработката на една връзка, `src/server.cpp`

4.2. Разпознаването

Функцията `classify` (листинг 2) получава прочетените октети и връща едно от три решения: TLS, некриптиран HTTP или непознато. Тя гледа само първия октет. Тя е `constexpr` и `noexcept`: не заделя памет, не вдига изключения и може да се изчисли още при компилиране. Тестовете използват това свойство, за да проверят част от поведението ѝ още при компилирането на теста.

```
36     [[nodiscard]] constexpr Wire classify(std::span<const std::uint8_t> first) noexcept
37     {
38         if (first.empty())
39         {
40             return Wire::Unknown;
41         }
42         const std::uint8_t octet = first.front();
43         if (octet == kContentHandshake)
44         {
45             return Wire::Tls;
46         }
```

```
47          if (octet >= 'A' && octet <= 'Z')
48          {
49              return Wire::Cleartext;
50          }
51          return Wire::Unknown;
52      }
```

Листинг 2: Разпознаването, `include/octet/classify.hpp`

Празен вход дава „непознато“, а не предположение. Това има значение при връзка, която е затворена, преди да изпрати нещо.

4.3. Връщане на прочетените октети

Разпознаването трябва да прочете поне един октет от сокета. Този октет обаче е и първият октет на ClientHello или на реда на заявката, и слоят след разпознаването се нуждае от него. Има два начина да не се изгуби.

Първият е флагът `MSG_PEEK` на `recv`: октетите се поглеждат, но остават в буфера на ядрото и следващото четене ги получава отново. Цената е второ системно извикване, което чете същите данни още веднъж.

Вторият начин е октетите да се прочетат наистина и после да се подадат на следващия слой, преди той да чете от сокета. `octet` използва втория. Класът `ReplayReader` (листинг 3) държи прочетените октети и при всяко четене първо връща тях. Едва когато свършат, той чете от сокета. Така никой слой след разпознаването не може да получи потока разбъркан или с пропуск.

```
38          // Returns replayed bytes first. Only once they are gone does it touch the
    ↪ source,
39          // so a caller can never see the stream out of order or with a gap.
40          net::IoResult read(std::span<std::uint8_t> out, const Deadline& deadline)
41          {
42              if (pos_ < pushback_.size())
43              {
44                  const std::size_t n = std::min(out.size(), pushback_.size() - pos_);
45                  std::memcpy(out.data(), pushback_.data() + pos_, n);
46                  pos_ += n;
47                  return {net::Io::Ok, n};
48              }
49              return source_(out, deadline);
50          }
```

Листинг 3: Четенето с връщане, `include/octet/replay.hpp`

При разпознаването сървърът иска до 2048 октета, а не само един. Това не

струва нищо повече, защото системното извикване е едно и също, а обикновено носи целия ClientHello или целия ред на заявката. При всички 2000 измерени TLS връзки с разпознаване във второто измерване на Windows първото четене е донесло целия ClientHello от 517 октета (същото е и в другите две измервания). Това означава, че при тези условия разпознаването не добавя нито едно системно извикване: OpenSSL получава ClientHello от буфера, а не от сокета. Условията са важни: клиентът и сървърът са на една машина и ClientHello пристига в един сегмент. Ако той пристигне на части, механизмът продължава да работи, както показва тестът в раздел 6, но тогава четенията са повече.

4.4. OpenSSL върху буфери в паметта

Обикновено OpenSSL чете и пише сокета сам. Тогава обаче не може да получи октетите, които разпознаването вече е прочело. Затова `octet` свързва OpenSSL с два буфера в паметта (BIO от вида `BIO_s_mem` [14]). Сървърът пълни входния буфер с октети от `ReplayReader` и изпразва изходния буфер към сокета.

Празен буфер в паметта по подразбиране означава край на потока, което OpenSSL би приел за прекъсната връзка. С `BIO_set_mem_eof_return(rbio, -1)` празният буфер вместо това връща „опитай пак“ [14]. Тогава OpenSSL отговаря с `SSL_ERROR_WANT_READ`, сървърът чете още октети и опитва отново.

Листинг 4 показва цикъла на ръкостискането. Всяка стъпка завършва с изпращане на всичко, което OpenSSL е произвело. При успех това е следващата група съобщения, а при провал е предупреждението. Предупреждение, което не е изпратено, оставя клиента да гадае защо връзката е прекъсната.

```
227     std::expected<void, std::string> Session::handshake(const Deadline& deadline)
228     {
229         for (;;)
230         {
231             ERR_clear_error();
232             const int rc = SSL_do_handshake(ssl_);
233             const int err = rc == 1 ? SSL_ERROR_NONE : SSL_get_error(ssl_, rc);
234             const std::string reason = (err == SSL_ERROR_NONE || err ==
    ↪ SSL_ERROR_WANT_READ) ? "" : last_error();
235
236             // Whatever OpenSSL produced goes out first: the next flight on success, the
```

```
            // alert on failure. An alert that is never sent leaves the client guessing.
            const auto sent = flush(deadline);

            if (err == SSL_ERROR_NONE)
            {
                if (sent != net::Io::Ok)
                {
                    return std::unexpected("handshake completed but the final flight was
    ↪  not sent");
                }
                return {};
            }
            if (err != SSL_ERROR_WANT_READ)
            {
                return std::unexpected("handshake failed: " + reason);
            }
            if (sent != net::Io::Ok)
            {
                return std::unexpected("could not send a handshake flight");
            }
            switch (feed(deadline))
            {
                case net::Io::Ok:
                    break;
                case net::Io::Timeout:
                    return std::unexpected("timeout");
                case net::Io::Closed:
                    return std::unexpected("peer closed during the handshake");
                case net::Io::Error:
                    return std::unexpected("socket error during the handshake");
            }
        }
    }
```

Листинг 4: Цикълът на ръкостискането, `src/tls.cpp`

Тази схема има и второ предимство: крайният срок остава в кода на сървъра. Всяко четене от сокета минава през `recv_some`, което чака само толкова, колкото е останало от срока. OpenSSL никога не блокира в собствено четене.

4.5. Един общ краен срок

Относителният срок, който започва отначало при всяко четене, не защитава от клиент, който изпраща по един октет точно преди изтичането му. Такъв клиент държи връзката отворена неограничено. Затова `octet` използва абсолютен срок (листинг 5): един момент във времето, от който всяко чакане изчислява колко е останало.

```
        // Milliseconds left, rounded up so that a wait never returns early, and clamped
        // to zero once the deadline has passed.
```

```
27          [[nodiscard]] int remaining_ms() const noexcept
28          {
29              const auto left = at_ - Clock::now();
30              if (left <= Clock::duration::zero())
31              {
32                  return 0;
33              }
34              const auto ms = std::chrono::ceil<std::chrono::milliseconds>(left).count();
35              return static_cast<int>(std::min<long long>(ms, 24LL * 60 * 60 * 1000));
36          }
```

Листинг 5: Оставащото време до срока, `include/octet/deadline.hpp`

Срокът започва при приемането на връзката и покрива и разпознаването, и цялото ръкостискане (листинг 1, ред 137). Клиент, който изпраща 0x16 и спира, е същата заплаха като клиент, който не изпраща нищо. Двата получават един и същ срок. След завършването на ръкостискането започва втори, отделен срок за заглавната част на HTTP заявката.

4.6. Настройките на TLS контекста

Контекстът на OpenSSL се създава веднъж при стартиране. Таблица 4 изброява настройките му и причината за всяка.

Таблица 4. Настройки на TLS контекста в `octet`

Настройка	Причина
минимална версия TLS 1.2, по избор 1.3	TLS 1.1 и по-старите не трябва да се договарят [5, раздел 3.1.1]
`SSL_OP_NO_RENEGOTIATION`	предоговарянето в TLS 1.2 е било обект на атаки [15], [11, раздел 2.10]; в TLS 1.3 то не съществува
`SSL_OP_NO_COMPRESSION`	компресията на ниво TLS отваря атаки като CRIME [11, раздел 2.6]; препоръката е да не се поддържа [5, раздел 3.3]
`SSL_OP_CIPHER_SERVER_PREFERENCE`	сървърът, а не клиентът, избира шифъра от общите [16]
строг ALPN	без общ приложен протокол връзката се отказва с предупреждение 120 [5, раздел 3.8]
проверка, че ключът съответства на сертификата	грешна конфигурация спира сървъра при стартиране, а не при първия клиент

Изборът на ALPN е в листинг 6. Ако клиентът предлага `http/1.1`, сървърът го избира. Ако не го предлага, строгият режим връща `SSL_TLSEXT_ERR_ALERT_FATAL`, което според документацията на OpenSSL означава, че няма общ протокол [17]. В раздел 6 се показва, че OpenSSL тогава наистина изпраща предупреждение 120. Клиент, който изобщо не изпраща ALPN разширение, се приема, защото няма какво да се откаже.

```
22          // RFC 7301 section 3.2: the server picks one protocol from the client's list.
    ↪ If
23          // it supports none of them it SHOULD answer with the no_application_protocol
24          // alert. SSL_TLSEXT_ERR_ALERT_FATAL is how OpenSSL is told to send that alert.
25          int alpn_select(SSL*, const unsigned char** out, unsigned char* outlen, const
    ↪ unsigned char* in,
26                          unsigned int inlen, void* arg)
27          {
28              const auto* policy = static_cast<const Context::AlpnPolicy*>(arg);
29              const auto choice = choose_alpn({in, inlen});
30              if (choice)
31              {
32                  *out = choice->data();
33                  *outlen = static_cast<unsigned char>(choice->size());
34                  return SSL_TLSEXT_ERR_OK;
35              }
36              return policy->strict ? SSL_TLSEXT_ERR_ALERT_FATAL : SSL_TLSEXT_ERR_NOACK;
37          }
```

Листинг 6: Изборът на приложен протокол, `src/tls.cpp`

4.7. Тестовият удостоверяващ орган

За тестовете и за измерването `octet` създава собствена малка инфраструктура на публичния ключ. Програмата `octet-mkcert` генерира два ключа ECDSA върху кривата P-256 и два сертификата. Първият е на удостоверяващия орган: основно ограничение CA:TRUE с дължина на пътя 0 и употреба на ключа само за подписване на сертификати и списъци за отнемане. Вторият е на сървъра, подписан от органа: CA:FALSE, употреба за цифров подпис, разширена употреба за удостоверяване на сървър и алтернативни имена `localhost` и `127.0.0.1`. Срокът е 30 дни.

Две нива вместо един самоподписан сертификат има по конкретна причина. Ако клиентът вярва директно на ключа на сървъра, не се проверява верига.

Тук клиентът вярва на органа и сървърът трябва да докаже името си със сертификат, подписан от него. Това е веригата, която проверява и браузърът. Тестовете използват стандартната проверка на клиента, включително проверката на името, без да я изключват. Ключовете са само за тестове и не се пазят в хранилището.

5. Анализ на сигурността

5.1. Модел на заплахите

Нападателят се разглежда като произволен клиент в мрежата. Той може да отваря връзки, да изпраща каквото пожелае, когато пожелае, и да не изпраща нищо. Нападател, който е между клиента и сървъра и може да променя трафика, се разглежда отделно в подраздел 5.5, защото там проблемът не е в кода, а в самата идея.

Активът е сървърът: неговите нишки, памет и процесорно време, както и коректността на решението кой протокол се говори. Целта на анализа е за всяка заплаха да посочи кое място в кода отговаря на нея и кой тест го проверява.

5.2. Заплахи и отговори

Таблица 5. Заплахи и отговорът на `octet`

Заплаха	Отговор в кода	Проверено от
Клиент се свързва и не изпраща нищо	общ краен срок от приемането; връзката се затваря при изтичането му	тест: затваряне при срока, докато други клиенти се обслужват
Клиент изпраща 0x16 и спира	същият срок покрива и ръкостискането	тест: затваряне при същия срок
Клиент изпраща по един октет точно преди изтичане	срокът е абсолютен и не се удължава от получени данни	по построение (листинг 5)
Много мълчащи връзки наведнъж	горна граница на едновременните връзки; над нея новите се затварят веднага	тест: третата връзка при граница 2

Заплаха	Отговор в кода	Проверено от
0x16, последван от безсмислени данни	разпознаването насочва към TLS, OpenSSL отхвърля записа	тест: няма некриптиран отговор
Произволни двоични данни или малки букви	„непознато“, затваряне без отговор	тестове за двоични данни и за `get`
Предупреждение (0x15) като първи запис	„непознато“, не се подава на OpenSSL	тест
Главна буква, която не е метод	анализаторът приема формата, сървърът отговаря 501 [1, раздел 15.6.2]	тест с метод `ZZZ`
Неправилен ред на заявката	отговор 400	тест
Огромна заглавна част на заявката	граница 8192 октета, отговор 431 [18]	тест; виж подраздел 5.4
Клиент без общ приложен протокол	фатално предупреждение 120	тест по октетите в мрежата
Клиент със стара версия	минимална версия на контекста	тест с клиент до TLS 1.2 срещу сървър с минимум 1.3
TLS клиент към сървър без сертификат	връзката се затваря; некриптиран отговор не би бил разбран	тест
Сървър с чуждо име	проверката е у клиента; сървърът не може да я направи вместо него	тест в паметта: грешка за несъвпадащо име

Няколко реда от таблицата изискват обяснение.

Защо непознатото се затваря без отговор. Връзка, която не започва нито с TLS запис, нито с HTTP метод, не говори протокол, който сървърът разбира. Всеки отговор би бил на език, който другата страна не е използвала. Освен това отговорът дава на сканиращия нападател информация за сървъра. Мълчаливото затваряне не дава нищо.

Защо разпознаването не проверява повече от един октет. Строг разпознавач би могъл да провери още версията на записа, границата на дължината и типа на съобщението за ръкостискане. Това изисква до шест октета и следователно чакане, докато пристигнат. Тези проверки обаче OpenSSL прави така или иначе и ги прави по-добре: той знае всички правила на протокола, а разпознавачът би знаел само някои от тях. Разпознаването в `octet` насочва, а не валидира. Тестът „0x16, последван от безсмислени данни“ проверява точно това: насочването е към TLS, но връзката не получава некриптиран отговор, защото OpenSSL отхвърля записа.

Защо проверката на името е у клиента. Сертификатът доказва, че сървърът притежава ключ за дадено име. Кое име е очаквал клиентът, знае само клиентът. Затова тестът със сървър с чуждо име се проваля в клиента, с грешка за несъвпадащо име, а не в сървъра.

5.3. Отклонение от препоръка в HTTP/1.1

RFC 9112 препоръчва сървърът да игнорира поне един празен ред, получен преди реда на заявката [8, раздел 2.2]. Препоръката е формулирана като SHOULD, тоест не е задължителна. `octet` не я следва: връзка, която започва с CR (0x0D), не започва с главна буква и се затваря като непозната. Това е съзнателен избор. Ако се приемат празни редове преди заявката, разпознаването вече не може да реши по първия октет и трябва да чака, докато празните редове свършат. Клиент, който изпраща празен ред преди заявката си, няма да бъде обслужен от `octet`.

5.4. Грешката, която тестовете откриха

Първата версия на анализатора на HTTP проверяваше границата от 8192 октета само докато не намери празния ред, който завършва заглавната част. Ако едно четене донесе едновременно октетите над границата и празния ред, проверката за празния ред се изпълняваше първа и заявката се приемаше. Тестът

на ниво функция не откри грешката, защото подава заявката по един октет. Интеграционният тест я откри: заявка с поле от 9000 октета получи отговор 200 вместо 431. Поправката проверява позицията на празния ред спрямо границата, а нов тест на ниво функция подава заявката на големи части, както я доставя мрежата.

5.5. Атаки с понижаване на защитата

Това е най-сериозното ограничение на темата и то не е в кода. RFC 9325 казва директно, че съчетаването на незащитена и защитена с TLS комуникация отваря пътя за SSL Stripping и подобни атаки, защото началото на комуникацията не е защитено от промяна и нападателят може да го манипулира така, че комуникацията да остане некриптирана [5, раздел 3.2]. SSL Stripping е описан в RFC 7457 [11, раздел 2.1]: нападател между клиента и сървъра премахва преминаването към TLS от некриптирания HTTP трафик.

Сървър, който обслужва и TLS, и некриптиран HTTP на един порт, е точно такова съчетание. Разпознаването по първия октет не създава уязвимостта: тя съществува и при два отделни порта, ако на порт 80 се обслужва съдържание. Но разпознаването прави некриптирания достъп до същата услуга възможен по подразбиране.

Стандартната защита е HTTP Strict Transport Security [19]. Сървърът изпраща по TLS заглавно поле, с което казва на браузъра за определен период да се свързва с него само по TLS. RFC 9325 изисква уеб сървърите да го поддържат [5, раздел 3.2]. `octet` в сегашния си вид не изпраща това заглавно поле и отговаря с едно и също съдържание и по двата транспорта. За производствена употреба правилната схема е следната: некриптираният път само пренасочва към адреса по TLS, а отговорите по TLS носят заглавното поле за HSTS. Тогава ползата от един порт остава (един сокет, една конфигурация), а некриптираният път не обслужва съдържание.

5.6. Какво не е проверено

Сървърът не е подлаган на автоматизирано подаване на случайни входни данни (fuzzing), нито на одит от трета страна. Кодът на OpenSSL се приема за коректен. Нишка за всяка връзка е проста схема, но тя ограничава броя на едновременните връзки до зададената граница. Сървър, който трябва да издържа на големи атаки за отказ от обслужване, би използвал асинхронен вход и изход. RFC 7457 отбелязва, че TLS сам по себе си е вектор за такива атаки, защото клиентът с малко усилие кара сървъра да извърши значителна изчислителна работа [11, раздел 2.13]. Това важи и за `octet`.

6. Проверка

Проверката има три нива. Тестовете на ниво функция проверяват частите поотделно. Тестът на TLS сесията провежда истински ръкостискания между клиент на OpenSSL и сесията на `octet` в един процес, без сокети. Интеграционният тест стартира изпълнимия файл и говори с него през истински сокети, така, както би говорил реален клиент. Всички тестове се стартират с `ctest` и минават на двете проверени платформи (табл. 6). Пълните записи от изпълненията са в хранилището, в папката `evidence`.

Таблица 6. Платформи, на които тестовете минават

	Windows	Linux
Система	Windows 11, 10.0.26200	Ubuntu 24.04 във WSL2, ядро 6.6
Компилатор	MSVC 19.51	GCC 14.2
OpenSSL	3.6.4 (vcpkg)	3.0.13 (от дистрибуцията)
Предупреждения при компилиране	няма при `/W4 /permissive-`	няма при `-Wall -Wextra -Wpedantic -Wconversion -Wshadow`

6.1. Тестове на ниво функция

Тестовете без мрежа са четири програми с общо 619 проверки, без външна рамка за тестване. Три от тях проверяват отделни части и са описани тук, а четвъртата провежда TLS сесии в паметта и е описана в следващия подраздел.

Тестът на разпознаването (538 проверки) минава през всичките 256 възможни стойности на първия октет и проверява, че точно една от тях води до TLS и точно 26 до HTTP. Отделно проверява всички стандартни методи на HTTP, началната последователност на HTTP/2, трите TLS записа, които не могат да отворят връзка, стария формат на SSL 2.0, празен вход, малки букви и водещ интервал. Проверява и че разпознаването гледа само първия октет: 0x16 в средата на заявка не променя решението.

Тестът на връщането на октети (12 проверки) проверява, че прочетените при разпознаването октети излизат първи, че източникът не се чете, докато те не свършат, и че TLS запис, пристигнал по един октет, излиза цял и в правилния ред.

Тестът на анализатора на HTTP (22 проверки) проверява валидни заглавни части и десет начина на неправилна: малки букви в метода, цел без наклонена черта, версия HTTP/2.0, двоен интервал, излишна дума, управляващ знак, твърде дълъг метод, сгъване на ред, интервал преди двоеточие и поле без двоеточие. Проверява и заглавна част, която надхвърля границата, подадена и по един октет, и на големи части.

6.2. TLS сесия в паметта

Тестът на TLS (47 проверки) провежда истински ръкостискания без мрежа и без втора нишка. Клиентът е обикновен клиент на OpenSSL с проверка на сертификата и на името. Хитростта е във функцията, от която сесията на сървъра чете: когато сървърът има нужда от октети и няма такива, тя изпълнява една стъпка от клиента. Клиентът обработва каквото сървърът е изпратил и

произвежда следващата си група съобщения. Двете страни се редуват в една нишка.

Първият октет на ClientHello се изважда и се подава обратно чрез `ReplayReader`, точно както прави сървърът след разпознаването. Проверени са:

- ръкостискане, при което сървърът получава ClientHello по един октет на четене; договорена е версия TLS 1.3 и протокол `http/1.1`, а клиентът потвърждава веригата до тестовия орган;
- обмен на данни в двете посоки след ръкостискането;
- клиент, който предлага само `h2`, получава предупреждение 120;
- клиент без ALPN и сървър в нестрог режим се приемат без избран протокол;
- клиент до TLS 1.2 се приема при минимум 1.2 и се отказва при минимум 1.3;
- клиент, който очаква друго име, отхвърля сертификата с грешка за несъвпадащо име.

6.3. Интеграционен тест

Интеграционният тест (35 проверки) стартира изпълнимия файл в шест конфигурации и говори с него с модула `ssl` на Python за TLS и с обикновени сокети за всичко останало. Проверките са групирани по конфигурации в таблица 7.

Проверката на отказа по ALPN заслужава отделно внимание. Python на Linux е свързан с по-стара версия на OpenSSL, която не разпознава името на това предупреждение и съобщава „unknown error“. Затова тестът не разчита на текста на грешката. Той изпраща ClientHello през обикновен сокет и търси в отговора точните седем октета:

```
15 03 03 00 02 02 78
```

Това е запис с ContentType 21 (предупреждение), версия 0x0303, дължина

Таблица 7. Конфигурации и проверки на интеграционния тест

Конфигурация	Какво се проверява
Един сокет, разпознаване, сертификат	TLS 1.3 с `http/1.1`; некриптиран HTTP на същия порт; без отговор на двоични данни, на малки букви и на предупреждение като първи запис; 501 за непознат метод; 400 за неправилен ред; 431 за огромна заглавна част; 0x16 с безсмислени данни не получава некриптиран отговор; отказ на клиент само с `h2` и точните октети на предупреждението; клиент без ALPN се обслужва; мълчащ клиент не пречи на другите и се затваря при срока; клиент, който спира след 0x16, се затваря при същия срок; записът на сървъра съдържа всички очаквани изходи
ClientHello по един октет	ръкостискането завършва; разпознаването е станало по част от ClientHello, а останалото е пристигало дълго след това
Без сертификат	TLS клиентът се затваря без некриптиран отговор; HTTP се обслужва
Минимум TLS 1.3	клиент до TLS 1.2 се отказва; клиент с TLS 1.3 се обслужва
Отделни слушатели	слушател само за TLS не отговаря на HTTP; слушател само за HTTP не завършва ръкостискане
Граница от две връзки	третата връзка се затваря веднага

2, ниво 2 (фатално) и описание 120 (no_application_protocol). Записът идва некриптиран, защото сървърът отказва още при обработката на ClientHello, преди ServerHello. Тази проверка минава еднакво и на двете платформи.

Тестът с ClientHello по един октет също е показателен. Клиентът изпраща всеки октет в отделен TCP сегмент с пауза от една милисекунда. Сървърът записва колко октета е донесло първото четене. Броят зависи от планирането на нишките, затова тестът проверява само, че е по-малък от сто, тоест че разпознаването е станало по малка част от ClientHello, и че до готовността на транспорта са минали над 300 милисекунди. Ръкостискането завършва успешно.

6.4. Какво остава непроверено

Конфигурацията за macOS съществува, но не е компилирана и изпълнена. Тестовете не включват автоматизирано подаване на случайни входни данни. Тестовете на Linux са изпълнени във WSL2, тоест във виртуална машина, а не на физически компютър с Linux.

7. Измерване на цената

Въпросът е колко струва разпознаването на една връзка в сравнение със сървър, който предварително знае какво ще пристигне. Отговорът има две части. Измерването от край до край показва дали цената се вижда отвън. Измерването в паметта показва колко е тя всъщност.

7.1. Постановка

Сравняват се четири варианта на сървъра, всеки като отделен процес на една и съща машина:

- TLS с разпознаване: един сокет, разпознаване включено, клиентът говори TLS;
- TLS с отделен слушател: разпознаването е изключено и сървърът започва ръкостискане веднага, без предварително четене;

- HTTP с разпознаване: същото, но клиентът говори некриптиран HTTP;
- HTTP с отделен слушател: разпознаването е изключено и сървърът подава връзката направо на анализатора на HTTP.

Всяка връзка носи една заявка и се затваря. Така цената за връзка се плаща веднъж на заявка, а не се разпределя между много заявки по една дълга връзка. Клиентът е на Python и използва стандартната проверка на сертификата и на името. Той отваря връзките една след друга с темп 100 в секунда. За всеки вариант и всяко повторение има по 20 загряващи връзки, които не влизат в резултата, и 400 измерени. Повторенията са 5, а редът на вариантите във всяко повторение се разбърква със записано начално число (20261103), за да не попадне бавно изместване в състоянието на машината само в един вариант.

Сървърът сам мери времената по монотонния си часовник и записва ред за всяка връзка. Записват се четири момента: стартирането на нишката на връзката, решението за протокола, готовността на транспорта и изпращането на отговора. Всички се отчитат от приемането на връзката.

7.2. Статистика

Сравнява се медианата на времето до изпращане на отговора. Ефектът на разпознаването е разликата на медианите, вариантът с разпознаване минус вариантът с отделен слушател. Доверителният интервал от 95% е получен с повторна извадка: 2000 пъти се теглят връзки с връщане, поотделно в рамките на всяко повторение, и се взема разликата на медианите. Интервалът е между 2,5-ия и 97,5-ия перцентил на тези разлики. Перцентилите на разпределенията се изчисляват по метода на най-близкия ранг. Скриптът за анализ е в хранилището и изходът му влиза в този документ автоматично, затова числата в текста не са преписвани на ръка.

Освен интервала се показва и разликата между медианите на отделните повторения. Ако повторенията се различават помежду си повече, отколкото е широк интервалът, интервалът подценява неопределеността и това трябва да се

каже.

7.3. Първото измерване и какво показа то

Първото измерване на Windows мереше времената от приемането на връзката. Разликите излязоха 280,0 µs за TLS (интервал от 81,0 до 496,0) и 83,5 µs за HTTP (от 7,0 до 162,5). Едно число в същите данни обаче показа, че тези разлики не измерват разпознаването. При HTTP с отделен слушател медианата на времето до решението беше 0,396 ms. Този вариант не чете нищо преди решението, така че времето до него трябваше да е почти нула. Това, което се измерваше, беше времето от приемането на връзката до момента, в който новата нишка започне да работи.

Стартирането на нишка е еднакво за всички варианти и не е част от механизма. Затова във второто измерване сървърът записва и момента на стартиране на нишката, и основният показател става времето от него до изпращането на отговора. Първото измерване е запазено в хранилището непроменено. То не е резултат, а измерването, което показа проблема.

7.4. Резултати

Таблици 8 и 9 показват времето от стартирането на нишката до изпращането на отговора, в милисекунди, за второто измерване на Windows и за Linux.

Таблица 8. Windows, време от стартирането на нишката до отговора (ms)

Рамо	*n*	p50	p90	p99	медиани по повторения
TLS, с разпознаване	2000	3,137	6,964	20,840	3,088 … 3,236
TLS, отделен слушател	2000	3,108	6,254	22,496	3,076 … 3,154
HTTP, с разпознаване	2000	0,126	0,436	20,424	0,124 … 0,128
HTTP, отделен слушател	2000	0,128	0,502	18,616	0,117 … 0,136

Таблица 9. Linux във WSL2, време от стартирането на нишката до отговора (ms)

Рамо	n	p50	p90	p99	медиани по повторения
TLS, с разпознаване	2000	2,169	13,636	27,076	2,084 . . . 4,008
TLS, отделен слушател	2000	2,163	10,390	16,126	2,109 . . . 9,543
HTTP, с разпознаване	2000	0,084	0,116	0,242	0,072 . . . 0,095
HTTP, отделен слушател	2000	0,079	0,110	0,183	0,068 . . . 0,090

Ефектът на разпознаването, измерен от стартирането на нишката, е следният.

- Windows, TLS: 28,5 µs, интервал от 5,0 до 53,0 µs, тоест 0,9% от медианата на отделния слушател.
- Windows, HTTP: −2,0 µs, интервал от −4,0 до 1,0 µs.
- Linux, TLS: 6,0 µs, интервал от −8,0 до 20,5 µs.
- Linux, HTTP: 5,0 µs, интервал от 2,0 до 8,0 µs, тоест 6,3% от медиана от 0,079 ms.

Последната колона на таблиците показва най-малката и най-голямата медиана сред отделните повторения. При TLS разликата между тях е по-голяма от самите ефекти, а при HTTP е от същия порядък. При TLS на Linux интервалът на ефекта включва нулата.

7.5. Цената в паметта

Когато първото четене донася целия ClientHello, разпознаването не добавя системно извикване. Остават заделяне на памет за буфера, копиране на ClientHello от буфера на разпознаването в буфера на OpenSSL, сравнение на първия октет и създаване на `ReplayReader`. Програмата `octet-micro` мери точно тази разлика, като замества системното извикване с копиране от фиксиран буфер и в двата пътя. Така то се съкращава по построение, а не чрез измерване. Резултатът е медиана от 15 кръга по 200000 повторения:

- Windows: 224,3 ns с разпознаване срещу 12,8 ns без, разлика 211,5 ns на връзка;

- Linux: 199,6 ns срещу 36,0 ns, разлика 163,6 ns на връзка.

7.6. Тълкуване

Цената на разпознаването в паметта е под една микросекунда на връзка на двете платформи. TLS връзката на същата машина отнема 3,108 ms на Windows и 2,163 ms на Linux (медиана при отделен слушател), тоест цената е под една хилядна от нея.

Измерването от край до край не може да различи цена от този порядък. Неговата разделителна способност е десетки микросекунди, колкото е и разликата между повторенията. Ефектите, които то показва, са в границите от няколко до около 30 микросекунди и не по-големи от 1% при TLS.

Ефектът при HTTP на Linux, 5,0 µs, е с интервал, който не включва нулата, и е с повече от порядък по-голям от цената в паметта. Причината за тази разлика не е установена. Възможни обяснения са различният ред на изчакване и четене при двата варианта или състоянието на кеша, но нито едно от тях не е проверено и работата не приписва числото на разпознаването.

Измерването от приемането на връзката е изкривено от стартирането на нишките. Във второто измерване на Windows медианите на това време бяха между 9,746 и 23,048 ms в различните варианти, а на Linux около 0,152 ms. Вариантите са изпълнявани в различни моменти и затова са получили различно натоварване на машината. Разликите, измерени от приемането, са 6813,5 µs при TLS и 4332,5 µs при HTTP. Те измерват машината, а не механизма, и не се цитират като цена на разпознаването.

Второто измерване на Windows отне 273 s, а първото 98 s, при еднакъв брой връзки. Причината за забавянето не е установена.

7.7. Ограничения на измерването

- Клиентът и сървърът са на една машина и говорят през локалния интерфейс. Мрежа няма.

- Клиентът е на Python. Той е по-бавен от сървъра и времето, което сървърът чака неговите съобщения, влиза в измерените времена и на четирите варианта.
- Linux е във WSL2, тоест във виртуална машина. Измерване на физически Linux няма.
- Сървърът използва нишка за всяка връзка. Сървър с асинхронен вход и изход би имал други времена.
- Първото измерване на Linux е отхвърлено: записите отиваха на диск на Windows през файловата система на WSL, която е бавна, и нишките на сървъра се натрупваха в очакване да запишат. То е запазено в хранилището под име, което казва това, и не е анализирано.

8. Изводи и заключение

Работата показа, че един октет е достатъчен, за да се реши дали нова TCP връзка е TLS или некриптиран HTTP. Основата е в стандартите: първият запис на TLS клиента е ръкостискане с ContentType 22, а HTTP методите се записват с главни латински букви. Тестът, който минава през всичките 256 стойности, потвърждава, че двете множества не се пресичат.

Самостоятелната реализация `octet` показва, че механизмът може да се изгради без загуба на данни и без да се отслабва TLS. Прочетените октети се връщат в TLS библиотеката чрез буфери в паметта. Ръкостискането се провежда от OpenSSL с минимална версия TLS 1.2, без предоговаряне, без компресия и със строго договаряне на приложния протокол. Един общ краен срок покрива и разпознаването, и ръкостискането, така че прозорецът преди ръкостискането не дава на бавния клиент повече време от обикновения TLS сървър. Тестовете минават на Windows и на Linux. Интеграционният тест откри реална грешка в границата на заглавната част на HTTP, която тестът на ниво функция не можеше да открие.

Цената на разпознаването е под една микросекунда на връзка в паметта и под разделителната способност на измерването от край до край. При тези условия разпознаването е практически безплатно и аргументът срещу него не може да е производителността.

Аргументът срещу него, който остава, е сигурността. Сървър, който обслужва защитен и незащитен транспорт на един порт, е съчетание, за което препоръките на IETF изрично предупреждават заради атаките с понижаване на защитата. Разпознаването по първия октет не създава този проблем, но и не го решава. Правилната употреба е некриптираният път само да пренасочва към TLS, а отговорите по TLS да носят HSTS. В сегашния си вид `octet` не прави това.

Естествените следващи стъпки са три. Първо, да се добавят пренасочването и HSTS, за да стане сървърът годен за реална употреба. Второ, да се подложи на автоматизирано подаване на случайни входни данни, най-вече анализаторът на HTTP и пътят на разпознаването. Трето, да се измери на физическа машина с Linux и през истинска мрежа, с клиент, който е по-бърз от сървъра, за да се стесни разделителната способност на измерването от край до край.

Списък с използваните съкращения

ALPN	Application-Layer Protocol Negotiation, договаряне на приложния протокол
ASCII	American Standard Code for Information Interchange
BIO	Basic Input/Output, абстракцията на OpenSSL за вход и изход
BCP	Best Current Practice, документ с препоръки на IETF
CA	Certificate Authority, удостоверяващ орган
CR	Carriage Return, октет 0x0D
CRIME	атака срещу компресията на ниво TLS, описана в RFC 7457, раздел 2.6
ECDSA	Elliptic Curve Digital Signature Algorithm
GCC	GNU Compiler Collection

HSTS	HTTP Strict Transport Security
HTTP	Hypertext Transfer Protocol
HTTPS	HTTP върху TLS
IETF	Internet Engineering Task Force
MSVC	Microsoft Visual C++
p50, p90, p99	50-и, 90-и и 99-и перцентил на разпределение
RFC	Request for Comments, документ от поредицата на IETF
SNI	Server Name Indication, указване на името на сървъра
SSH	Secure Shell
SSL	Secure Sockets Layer, предшественикът на TLS
TCP	Transmission Control Protocol
TLS	Transport Layer Security
WSL2	Windows Subsystem for Linux, версия 2
X.509	стандарт за формата на сертификатите с публичен ключ

Списък с използваните литературни източници

[1] R. Fielding, M. Nottingham, and J. Reschke, “HTTP semantics.” RFC 9110, IETF, 2022.

[2] E. Rescorla, “The transport layer security (TLS) protocol version 1.3.” RFC 8446, IETF, 2018.

[3] S. Friedl, A. Popov, A. Langley, and E. Stephan, “Transport layer security (TLS) application-layer protocol negotiation extension.” RFC 7301, IETF, 2014.

[4] A. Tsvetanov, “octet: TLS and cleartext HTTP on one listening socket, chosen by the first octet.” Source repository, 2026.

[5] Y. Sheffer, P. Saint-Andre, and T. Fossati, “Recommendations for secure use of transport layer security (TLS) and datagram transport layer security (DTLS).” RFC 9325, BCP 195, IETF, 2022.

[6] T. Dierks and E. Rescorla, “The transport layer security (TLS) protocol version 1.2.” RFC 5246, IETF, 2008.

[7] D. Cooper, S. Santesson, S. Farrell, S. Boeyen, R. Housley, and W. Polk, “Internet X.509 public key infrastructure certificate and certificate revocation list (CRL) profile.” RFC 5280, IETF, 2008.

[8] R. Fielding, M. Nottingham, and J. Reschke, “HTTP/1.1.” RFC 9112, IETF, 2022.

[9] M. Thomson and C. Benfield, “HTTP/2.” RFC 9113, IETF, 2022.

[10] S. Turner and T. Polk, “Prohibiting secure sockets layer (SSL) version 2.0.” RFC 6176, IETF, 2011.

[11] Y. Sheffer, R. Holz, and P. Saint-Andre, “Summarizing known attacks on transport layer security (TLS) and datagram TLS (DTLS).” RFC 7457, IETF, 2015.

[12] OpenVPN Inc., “Reference manual for OpenVPN 2.6, option --port-share.” Online, 2026. Accessed 7 Oct 2026.

[13] Y. Rutschle, “sslh: Applicative protocol multiplexer.” Source repository and README, 2026. Accessed 7 Oct 2026.

[14] The OpenSSL Project, “BIO_s_mem, OpenSSL 3.0 manual.” Online, 2026. Accessed 3 Nov 2026.

[15] E. Rescorla, M. Ray, S. Dispensa, and N. Oskov, “Transport layer security (TLS) renegotiation indication extension.” RFC 5746, IETF, 2010.

[16] The OpenSSL Project, “SSL_CTX_set_options, OpenSSL 3.0 manual.” Online, 2026. Accessed 3 Nov 2026.

[17] The OpenSSL Project, “SSL_CTX_set_alpn_select_cb, OpenSSL 3.0 manual.” Online, 2026. Accessed 3 Nov 2026.

[18] M. Nottingham and R. Fielding, “Additional HTTP status codes.” RFC 6585, IETF, 2012.

[19] J. Hodges, C. Jackson, and A. Barth, “HTTP strict transport security (HSTS).” RFC 6797, IETF, 2012.